

AI Copilot Usage Log

Project: OzZoo – Object-Oriented Virtual Zoo Simulator

Author: Manan Sharma | Student ID: BSIT 2023-2026, VSIT Mumbai

Date: March 2026 | Tool: GitHub Copilot / GPT-based AI assistant

This log documents my use of an AI coding assistant (GitHub Copilot / GPT) during the development of **OzZoo**, a Python object-oriented virtual zoo simulator. The project implements 13 classes across a four-level inheritance hierarchy, a Singleton design pattern (Finance), and a composition-based orchestrator (Zoo).

Each entry below records: the **key prompt** I submitted, a representative **AI output snippet**, and my **analysis** explaining why the prompt was used, how helpful the output was, what I modified, and what I learned.

Colour Legend	Blue = Key Prompt	Grey = AI Output	Green = Analysis	Amber = Modifications & Learning
---------------	-------------------	------------------	------------------	----------------------------------

Entry 1 Project Architecture Design

Purpose: Before writing any code, I needed a clear OOP blueprint: which classes to create, how they relate, and what design patterns to apply.

KEY PROMPT

Prompt: "Design a Python OOP zoo simulator called OzZoo. Identify all classes needed, their inheritance hierarchy, and any design patterns. Include animals (mammals, birds, reptiles), habitats, finance, and a top-level Zoo class."

AI OUTPUT SNIPPET

```
# Suggested class hierarchy
Animal (ABC) ← abstract base
Mammal(Animal) ← intermediate
Lion(Mammal), Elephant(Mammal) ← concrete
Bird(Animal) ← intermediate
Eagle(Bird), Parrot(Bird) ← concrete
Reptile(Animal) ← intermediate
Crocodile(Reptile), Gecko(Reptile) ← concrete

Habitat - standalone; owns animal roster + environment logic
Finance - Singleton (only ONE ledger per zoo)
Zoo - orchestrator; uses Habitat and Finance via composition
```

YOUR ANALYSIS

The AI immediately produced a clean three-tier inheritance tree and correctly recommended the Singleton pattern for Finance and composition for Zoo. This saved significant up-front design time and gave a solid, testable structure that maps directly to real OOP principles taught in the course.

MODIFICATIONS & LEARNING

I accepted the hierarchy as-is but added the Zoo class as the top-level orchestrator, which the AI had not explicitly included. I also decided to add a Habitat class with climate validation — an idea prompted by the AI's mention of 'environment logic'. Learned: AI excels at high-level design scaffolding.

Entry 2 Abstract Base Class with Encapsulation

Purpose: Implement the Animal ABC with properly encapsulated private attributes and @property descriptors so subclasses cannot bypass validation.

KEY PROMPT

Prompt: "Write a Python abstract base class Animal with private attributes __health, __hunger, __happiness using double-underscore name mangling. Add @property getters and setters that clamp values between 0 and 100. Add abstract methods sound() and diet(). Add concrete methods feed(), play(), and receive_treatment()."

AI OUTPUT SNIPPET

```
from abc import ABC, abstractmethod

class Animal(ABC):
    _ticket_price: float = 10.0

    def __init__(self, name, species, health=100, hunger=0, happiness=80):
        self.__health = health
        self.__hunger = hunger
        self.__happiness = happiness

    @property
    def health(self) -> int:
        return self.__health

    @health.setter
    def health(self, value: int) -> None:
        self.__health = max(0, min(100, value))

    @abstractmethod
    def sound(self) -> str: ...

    def feed(self, amount=20) -> str:
        self.hunger -= amount
        self.happiness += 5
        return f'{self.name} has been fed.'
```

YOUR ANALYSIS

The AI generated exactly the pattern needed: double-underscore mangling prevents direct access from outside the class, while the clamping setter (max/min) ensures stats never go out of range. The output was immediately usable and matched the encapsulation style required by the assignment.

MODIFICATIONS & LEARNING

I added a public `ticket_price` property wrapping the protected `_ticket_price` class variable — the AI had left it as a bare class attribute, which would allow external code to access `_ticket_price` directly. I also added `__name` and `__species` as private attributes (the AI had left them public). Learned: always check that ALL attributes are properly encapsulated, not just the ones explicitly listed in the prompt.

Entry 3 Intermediate Inheritance Classes

Purpose: Create Mammal, Bird, and Reptile as intermediate classes that extend Animal and add group-specific attributes and behaviours before the concrete leaf classes.

KEY PROMPT

Prompt: "Create three intermediate Python classes: Mammal(Animal), Bird(Animal), and Reptile(Animal). Each should add group-specific attributes (`fur_colour`, `wingspan_cm`, `scale_colour` etc.) and one new method (`groom`, `fly`, `bask`). Reptile should set `is_warm_blooded = False` as a class constant."

AI OUTPUT SNIPPET

```
class Mammal(Animal):
    is_warm_blooded: bool = True

    def __init__(self, name, species, fur_colour='brown', **kwargs):
        super().__init__(name, species, **kwargs)
        self.fur_colour = fur_colour

    def groom(self) -> str:
        self.happiness += 10
        return f'{self.name} was groomed.'

class Reptile(Animal):
    is_warm_blooded: bool = False

    def bask(self, minutes=30) -> str:
        self.health += minutes // 10
        return f'{self.name} basked for {minutes} min.'
```

YOUR ANALYSIS

The AI correctly used `super().__init__()` with `**kwargs` to forward arguments up the chain — a pattern that is easy to get wrong. The `is_warm_blooded` class constant cleanly distinguishes warm- and cold-blooded animals without requiring runtime checks. The output required minimal changes.

MODIFICATIONS & LEARNING

I added default implementations of `sound()` and `diet()` to each intermediate class so they are not accidentally left abstract — the AI had not included these, which would have made Mammal, Bird, and Reptile abstract too (unable to be instantiated for quick testing). Learned: always verify that intermediate classes satisfy the ABC contract.

Entry 4 Concrete Animal Classes

Purpose: Implement six concrete leaf classes — Lion, Elephant, Eagle, Parrot, Crocodile, Gecko — each with a unique species, ticket price, sound, diet, and at least one species-specific method.

KEY PROMPT

Prompt: "Write a Python class Lion(Mammal) with species='Panthera leo', fur_colour='golden', ticket_price=\$25, sound()='ROAR', diet()='carnivore', and a hunt() method that increases hunger by 20 and happiness by 10. Follow the same pattern for Elephant, Eagle, Parrot, Crocodile, and Gecko."

AI OUTPUT SNIPPET

```
class Lion(Mammal):
    _ticket_price: float = 25.0

    def __init__(self, name, is_male=True, mane_length='medium', **kwargs):
        super().__init__(name=name, species='Panthera leo',
            fur_colour='golden', **kwargs)
        self.is_male = is_male
        self.mane_length = mane_length

    def sound(self) -> str: return 'ROAR'
    def diet(self) -> str: return 'carnivore'

    def hunt(self) -> str:
        self.hunger += 20
        self.happiness += 10
        return f'{self.name} went on a hunt!'
```

YOUR ANALYSIS

The AI produced consistent, idiomatic code for all six concrete classes in one pass. Hardcoding species, fur_colour, and ticket_price inside __init__ keeps each class self-contained. The **kwargs pattern ensures future attributes added to Animal or Mammal automatically flow through without changing the leaf classes.

MODIFICATIONS & LEARNING

For Parrot, the AI generated talk() without randomness — I changed it to use random.choice(self.vocabulary) to make interactions more dynamic. For Gecko, I added a can_regrow_tail = True class constant as a fun biological detail. Learned: the AI gives correct structure but domain-specific details need human input.

Entry 5 Habitat Class with Climate Validation

Purpose: Build the Habitat class to manage animal rosters, enforce capacity limits, validate climate and enclosure type, and run a daily simulation loop.

KEY PROMPT

Prompt: "Write a Python Habitat class with attributes: name, climate (validated against a tuple of allowed values), enclosure_type, capacity, area_sqm. It should have add_animal() and remove_animal() methods. Add a simulate_day() method that increases every animal's hunger by 5, gives reptiles in arid/tropical climates +3 health, and gives aquatic habitat animals +5 happiness."

AI OUTPUT SNIPPET

```

class Habitat:
    VALID_CLIMATES = ('tropical','arid','temperate','aquatic','arctic')
    VALID_ENCLOSURES = ('open','cage','tank','aviary','terrarium')

    def __init__(self, name, climate='temperate', ...):
        if climate not in self.VALID_CLIMATES:
            raise ValueError(f'Invalid climate {climate!r}.')
        ...

    def simulate_day(self) -> list[str]:
        from ozzoo.animals.reptile import Reptile
        log = []
        for animal in self._animals:
            animal.hunger += 5
            if isinstance(animal, Reptile) and self.climate in ('arid','tropical'):
                animal.health += 3
            if self.climate == 'aquatic':
                animal.happiness += 5
        return log

```

YOUR ANALYSIS

The AI correctly placed validation in `__init__` using class-level tuple constants rather than magic strings scattered through methods. The deferred import of `Reptile` inside `simulate_day()` avoids a circular import — a subtlety the AI handled without being asked. The `simulate_day()` logic matches the specification exactly.

MODIFICATIONS & LEARNING

I added `log.append()` calls inside the loop so the method returns a human-readable list of events (the AI returned an empty log). I also added a read-only `animals` property (returning `list(self._animals)`) so callers cannot mutate the internal list directly. Learned: AI often omits logging and copy-safety; always review return values carefully.

Entry 6 Finance Singleton Pattern

Purpose: Implement a Finance class that guarantees only ONE instance exists across the entire application, using Python's `__new__` and an `_initialised` guard.

KEY PROMPT

Prompt: "Write a Python Finance class implementing the Singleton pattern using `__new__`. It should have private attributes `__balance` (starting at 50000), `__revenue`, and `__expenses`, all exposed via `@property`. Include methods: `charge_admission()`, `pay_food_bill()`, `pay_vet_bill()`, `record_expense()`, `record_income()`, and `report()`. Add a class method `reset()` for test isolation."

AI OUTPUT SNIPPET

```

class Finance:
    _instance: Finance | None = None
    _initialised: bool = False

    def __new__(cls) -> Finance:
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self) -> None:
        if self._initialised: return
        self.__balance = 50_000.0
        self.__revenue = 0.0
        self.__expenses = 0.0
        self._initialised = True

    @classmethod
    def reset(cls) -> None:
        cls._instance = None
        cls._initialised = False

```

YOUR ANALYSIS

The AI produced a textbook Singleton with the `_initialised` guard — critical to prevent `__init__` from resetting the balance on every `Finance()` call. The `reset()` classmethod is an important addition for unit testing that many Singleton examples omit. The output was used verbatim with no structural changes.

MODIFICATIONS & LEARNING

No structural changes were needed. I did verify the singleton by adding `'f1 = Finance(); f2 = Finance(); assert f1 is f2'` to `main.py` — the AI suggested this test pattern when I asked. Learned: the Singleton pattern in Python is clean when `__new__` and `_initialised` work together; memorise this idiom.

Entry 7 Zoo Orchestrator (Composition over Inheritance)

Purpose: Build the top-level Zoo class that wires Habitats and Finance together using composition, providing high-level operations like `admit_visitors()` and `simulate_day()`.

KEY PROMPT

Prompt: "Write a Zoo class that owns a list of Habitat objects and a reference to the Finance singleton. Do NOT use inheritance — use composition. Include `add_habitat()`, `place_animal()`, `admit_visitors(count)`, `simulate_day()`, and `full_report()`. `admit_visitors` should charge each animal's `ticket_price` via Finance."

AI OUTPUT SNIPPET

```

class Zoo:
    def __init__(self, name, location='Sydney, Australia'):
        self.name = name
        self.location = location
        self._habitats: list[Habitat] = []
        self._finance: Finance = Finance()

    def admit_visitors(self, count=1) -> list[str]:
        log = []
        for habitat in self._habitats:
            for animal in habitat.animals:
                msg = self._finance.charge_admission(
                    animal._ticket_price, count)
                log.append(f'[{habitat.name}] {animal.name}: {msg}')
        return log

```

YOUR ANALYSIS

The AI correctly modelled Zoo as a composition class with no custom parent, and naturally used Finance() to obtain the singleton rather than passing it in. The full_report() method it generated aggregates all habitat reports and the finance report into a single printable string — very useful for demos.

MODIFICATIONS & LEARNING

The AI accessed animal._ticket_price (a protected attribute) directly inside Zoo. I changed this to animal.ticket_price (the public property added to Animal) to respect encapsulation. I also moved simulate_day() to deduct daily operating costs per habitat (\$500 each) — the AI had not included this. Learned: always check that cross-class access respects the access modifiers you defined.

Entry 8 Debugging — Encapsulation Violation Fix

Purpose: During code review the AI flagged that Zoo was directly reading animal._ticket_price (a protected attribute), breaking encapsulation. I used the AI to find the cleanest fix.

KEY PROMPT

Prompt: "In my Zoo class I am accessing animal._ticket_price directly. How do I fix this to respect encapsulation without changing the existing _ticket_price class variable on Animal and its subclasses?"

AI OUTPUT SNIPPET

```

# Add a public property to Animal:
@property
def ticket_price(self) -> float:
    return self._ticket_price

# Then in Zoo, replace:
# animal._ticket_price
# with:
# animal.ticket_price

# This keeps _ticket_price as the class-level override point
# while exposing a clean public API.

```

YOUR ANALYSIS

This was a targeted debugging session. The AI identified the root cause immediately and suggested adding a thin property wrapper — the minimal change that fixes the encapsulation violation without requiring any refactor of the existing subclass `_ticket_price` overrides. The suggestion was correct and clean.

MODIFICATIONS & LEARNING

I implemented the fix exactly as suggested (added `ticket_price` property to `Animal`, updated `Zoo.admit_visitors`). No further changes were needed. Learned: when encapsulation is violated across class boundaries, a single property getter is often the smallest correct fix. AI is particularly good at these surgical 'find the minimal change' queries.

Summary: AI Contribution vs. Human Modifications

Entry	AI Contribution	Key Human Modification
1 – Architecture	Full class hierarchy + pattern selection	Added Zoo orchestrator class
2 – Animal ABC	Encapsulation, properties, abstract methods	Added <code>ticket_price</code> property; made <code>__name</code> private
3 – Intermediate	<code>super()</code> chain with <code>**kwargs</code> ; class constants	Added default <code>sound()/diet()</code> implementations
4 – Concrete Animals	Consistent structure across 6 classes	Added randomness to Parrot; Gecko biology detail
5 – Habitat	Climate validation; deferred import; <code>simulate_day()</code>	Added log messages; copy-safe animals property
6 – Finance	Singleton <code>__new__</code> + guard; <code>reset()</code> for tests	Used verbatim; verified with assert test
7 – Zoo	Composition design; <code>full_report()</code>	Fixed <code>_ticket_price</code> access; added operating costs
8 – Debug Fix	Minimal property-wrapper solution	Implemented fix across Animal + Zoo

Overall Reflection: The AI copilot was most valuable for boilerplate generation (class scaffolding, property descriptors, design patterns) and for targeted debugging ('find the minimal fix'). Human judgement was essential for domain details, logging completeness, copy-safety, and enforcing consistent encapsulation across class boundaries. Critical engagement — verifying every output before accepting it — was the key skill practised throughout this project.